

Ciência da Computação – Programação Distribuída e Paralela		
Aluno (a):		
Turma:	Bimestre: 2º	Data da Entrega:

Engenharia de Contexto e “Harness Engineering” aplicada à Programação Distribuída e Paralela.

Instruções da atividade avaliativa:

- Esta atividade deve ser desenvolvida em até 3 alunos.
- A atividade deve ser entregue no tempo estabelecido.
- A identificação de **plágio resulta em anulação da prova** (nota 0).
- **TODA ferramenta ou IA utilizada deve ser justificada.**

1. Apresentação e Contextualização

A engenharia de contexto (context engineering) e o desenvolvimento de harnesses, estruturas que orquestram chamadas a modelos de linguagem com ferramentas, memória e fluxos de controle, vêm se tornando habilidades essenciais no desenvolvimento de software moderno. Quando combinadas com computação distribuída e paralela, surgem desafios particularmente interessantes: como paralelizar chamadas a múltiplos agentes de IA, como distribuir cargas de inferência entre nós, como projetar contextos eficientes em ambientes com latência variável e múltiplos workers cooperando.

Esta atividade propõe que cada equipe construa um sistema distribuído ou paralelo que utilize modelos de linguagem como componentes de processamento, aplicando os conceitos vistos em sala (concorrência, sincronização, troca de mensagens, MapReduce, balanceamento de carga, tolerância a falhas) sobre infraestrutura AWS Academy.

2. Objetivos de Aprendizagem

Ao final desta atividade, o aluno deverá ser capaz de:

- Projetar e implementar um sistema que combine paralelismo ou distribuição com orquestração de modelos de linguagem.

- Aplicar princípios de engenharia de contexto, como estruturação de prompts, gestão de janela de contexto, definição de ferramentas (tools) e estratégias de memória, em cenários multi-agente ou multi-worker.
- Avaliar trade-offs entre custo, latência e qualidade ao paralelizar inferências de IA.
- Utilizar serviços AWS para construir uma arquitetura escalável e tolerante a falhas.
- Documentar decisões técnicas e analisar empiricamente os resultados obtidos.

3. Formação de Equipes e Prazo

As equipes deverão ter até 3 integrantes. Equipes menores são permitidas, porém o escopo da entrega permanece o mesmo. A definição de equipes e tema escolhido deve ser comunicada ao professor na primeira semana da atividade.

4. Requisitos Técnicos Mínimos

Independentemente do tema escolhido, a solução entregue deve atender obrigatoriamente a todos os requisitos abaixo:

4.1 Paralelismo ou Distribuição Real

- Pelo menos dois nós ou processos cooperando entre si, configurando paralelismo real e não apenas operações assíncronas em um único processo.
- Uso de algum mecanismo de comunicação distribuída, como filas SQS, tópicos SNS, sockets, gRPC, Lambda assíncrono, ou similares.

4.2 Integração com Modelo de Linguagem

- Integração com pelo menos um modelo de linguagem, podendo ser via Amazon Bedrock (caso disponível na conta AWS Academy) ou via SLM (Small Language Model) auto-hospedado.

4.3 Camada Explícita de Engenharia de Contexto

- System prompts versionados e documentados em arquivos separados do código de aplicação.
- Estratégia clara de chunking ou particionamento de entrada quando aplicável.
- Gestão explícita de histórico ou memória entre chamadas, quando o tema exigir.
- Definição de ferramentas (tools) ou function calling, quando aplicável ao tema.

4.4 Métricas e Observabilidade

- Coleta sistemática de métricas, incluindo latência por requisição, throughput agregado, tokens consumidos e taxa de erro.
- Logs estruturados que permitam reconstruir o fluxo de execução.

4.5 Tolerância a Falhas

- Implementação de retry com backoff em chamadas a serviços externos.
- Tratamento de mensagens com falha, por exemplo via Dead Letter Queue.
- Estratégia de fallback documentada para os pontos críticos do sistema.

5. Temas Disponíveis

Cada equipe deve escolher exatamente um dos temas abaixo. Os temas foram pensados para explorar diferentes aspectos da disciplina e diferentes desafios de engenharia de contexto. Equipes que tenham interesse em propor um tema próprio podem fazê-lo, desde que aprovado previamente pelo professor.

Tema 1. Pipeline distribuído de análise de documentos

Construir um sistema que recebe lotes de PDFs (contratos, artigos científicos, decisões judiciais ou similares) e produz resumos estruturados, extração de entidades e classificação. O paralelismo aparece no particionamento dos documentos entre workers e no processamento de seções de um mesmo documento em paralelo (fase de map), com posterior consolidação dos resultados (fase de reduce). O desafio de engenharia de contexto está em projetar prompts que mantenham coerência entre chunks, lidar com referências cruzadas entre seções e definir um schema de saída estruturada (JSON) consistente.

Conceitos da disciplina explorados: MapReduce, particionamento de dados, sincronização de resultados parciais, agregação.

Tema 2. Sistema multi-agente para resolução colaborativa

Implementar uma arquitetura onde múltiplos agentes especializados (por exemplo: planejador, executor, crítico, revisor) rodam em processos ou containers separados e se comunicam via filas para resolver problemas complexos, como geração de código, análise de dados ou planejamento de viagem. Cada agente possui um contexto e system prompt distinto e ferramentas próprias. O desafio é orquestrar a colaboração sem que agentes fiquem em estados de espera mútua indefinida.

Conceitos da disciplina explorados: produtor consumidor, deadlock e starvation, protocolos de coordenação simplificados, mensageria.

Tema 3. Web scraping inteligente paralelo com extração via IA

Sistema distribuído de coleta de dados na web onde workers paralelos baixam páginas e um pool de extratores baseados em modelo de linguagem interpreta o conteúdo HTML ou texto para extrair dados estruturados, como preços de produtos, vagas de emprego, notícias ou dados públicos. A engenharia de contexto entra na criação de prompts robustos a variações de layout e na gestão eficiente de tokens, dado que documentos HTML podem ser muito extensos.

Conceitos da disciplina explorados: pool de workers, rate limiting distribuído, balanceamento de carga, backpressure.

Tema 4. Tradutor e localizador distribuído de grandes volumes de texto

Receber um corpus extenso (livros em domínio público, documentação técnica, datasets textuais) e produzir tradução paralela com consistência terminológica. O grande desafio de contexto é a coerência: um termo técnico traduzido na seção 1 deve permanecer consistente na seção 200, mesmo que tenha sido processado por workers diferentes.

Exige construir um glossário distribuído compartilhado entre os workers, possivelmente em DynamoDB ou Redis no ElastiCache.

Conceitos da disciplina explorados: memória compartilhada distribuída, exclusão mútua, consistência eventual, sincronização.

Tema 5. Sistema de Q&A sobre base de conhecimento (RAG distribuído)

Implementar um pipeline RAG (Retrieval-Augmented Generation) onde a indexação dos documentos é feita em paralelo, com embeddings calculados por múltiplos workers, e as consultas são atendidas por um pool de instâncias de modelo de linguagem. A engenharia de contexto envolve estratégias de chunking, re-ranking de resultados recuperados e montagem do contexto final dentro do limite de tokens disponível.

Conceitos da disciplina explorados: paralelismo de dados na indexação, paralelismo de tarefas no atendimento, cache distribuído.

Tema 6. Moderação de conteúdo paralela em tempo (quase) real

Stream de mensagens (simulado via SQS ou Kinesis) sendo classificado por workers paralelos quanto a toxicidade, spam e categorias customizadas. Envolve engenharia de contexto para criar classificadores baseados em modelo de linguagem com few-shot examples cuidadosamente escolhidos, além dos desafios de paralelismo na manutenção de contadores agregados (mensagens por usuário, por tópico, por janela de tempo).

Conceitos da disciplina explorados: stream processing, sliding windows, contadores distribuídos, race conditions.

Tema 7. Gerador paralelo de testes e análise de código

Sistema que recebe um repositório de código, distribui os arquivos entre workers, e cada worker usa um modelo de linguagem para gerar testes unitários, identificar code smells ou produzir documentação técnica. Um agregador consolida os resultados e detecta inconsistências entre as análises de diferentes arquivos.

Conceitos da disciplina explorados: paralelismo embaraçoso (embarrassingly parallel), agregação, detecção de conflitos, particionamento por arquivo.

Tema 8. Simulador de exame de agentes para benchmarking

Construir um framework que executa N instâncias de um mesmo agente baseado em modelo de linguagem em paralelo sobre um benchmark (por exemplo GSM8K, HumanEval simplificado ou um benchmark customizado pela equipe), com diferentes estratégias de prompting (zero-shot, chain-of-thought, self-consistency com voto majoritário). O foco é a infraestrutura de execução paralela e a comparação empírica de estratégias de contexto.

Conceitos da disciplina explorados: paralelismo de tarefas, agregação por votação, análise estatística de resultados distribuídos.

6. Infraestrutura AWS Sugerida

As equipes podem combinar livremente os serviços disponíveis no AWS Academy. A tabela abaixo resume os serviços mais úteis para esta atividade. Antes de começar, cada equipe deve verificar quais desses serviços estão de fato habilitados em sua conta de laboratório, pois isso varia entre turmas.

Serviço	Uso Sugerido
EC2	Workers persistentes, hospedagem de modelos SLM, servidores de orquestração.
Lambda	Workers serverless de curta duração, processamento orientado a eventos.
SQS / SNS	Mensageria assíncrona entre componentes, fan-out e desacoplamento.
DynamoDB	Estado compartilhado, contadores distribuídos, tabelas de checkpoint.
S3	Armazenamento de inputs, outputs e artefatos de processamento.
Bedrock	Acesso a modelos como Claude, Llama, Titan (verificar disponibilidade na conta).
ECS / Fargate	Containers para workers e agentes em execução contínua.
CloudWatch	Coleta de métricas, logs estruturados e alarmes.
ElastiCache	Cache distribuído, em especial Redis para o tema do glossário.

7. Plano Alternativo: Uso de SLM caso o Bedrock não esteja disponível

Algumas configurações do AWS Academy não permitem o uso do Amazon Bedrock. Caso a equipe identifique essa limitação, é totalmente viável e pedagogicamente rico substituir o Bedrock por um SLM (Small Language Model) auto-hospedado em EC2. Esta seção descreve as alternativas.

7.1 Por que SLMs funcionam bem para esta atividade

SLMs são modelos com até aproximadamente 10 bilhões de parâmetros que rodam em uma única GPU ou até em CPU para os menores, entregando entre 80 e 90 por cento da qualidade de modelos grandes em tarefas focadas, com custo de inferência significativamente menor. Para fins didáticos de Programação Distribuída e Paralela, SLMs são ideais: a equipe controla totalmente a infraestrutura de inferência, o que abre espaço para experimentar com paralelização, batching e balanceamento de carga de forma muito mais visível do que ao chamar uma API gerenciada.

7.2 Modelos SLM Recomendados

A escolha do modelo depende do tipo de tarefa do tema escolhido. Sugestões testadas e amplamente disponíveis em maio de 2026:

Modelo	Parâmetros	Pontos fortes	Bom para os Temas
Phi-4 (Microsoft)	3.8B	Forte em raciocínio e matemática.	2, 7, 8
Gemma 3 (Google)	2B / 9B / 27B	Bom equilíbrio qualidade e tamanho, multimodal.	1, 3, 6
Qwen 2.5 / Qwen 3	0.5B a 7B	Forte multilíngue (PT-BR), contexto até 32K.	1, 4, qualquer com PT
Llama 3.2 (Meta)	1B / 3B / 8B	Edge e mobile, function calling, bem documentado.	2, 3, 7
Mistral 7B Instruct	7B	Apache 2.0, sólido para uso geral.	Qualquer tema
SmolLM3 (HF)	3B	Apache 2.0, raciocínio dual mode.	Experimentação

7.3 Como Hospedar no AWS Academy

Opção A. Ollama em EC2 com GPU (recomendada)

Ollama é a forma mais simples de servir SLMs. Expõe uma API HTTP em localhost:11434 e cuida automaticamente de quantização, gerenciamento de memória e troca de modelos.

Cenário	Instância EC2	VRAM	Modelos viáveis
Modelos pequenos (1B a 3B)	g4dn.xlarge	16 GB (T4)	Phi-4, Gemma 3 2B, Llama 3.2 3B, SmolLM3
Modelos médios (7B a 9B)	g4dn.xlarge ou g5.xlarge	16 a 24 GB	Mistral 7B, Llama 3.1 8B, Qwen 2.5 7B
Modelos um pouco maiores	g5.2xlarge	24 GB (A10G)	Qualquer SLM até 13B com Q4

Importante: o AWS Academy pode ter limites de quota e nem sempre permite instâncias GPU. Antes de qualquer coisa, a equipe deve verificar quais tipos de instância e quais regiões estão disponíveis na conta Academy específica. Caso GPU não esteja disponível, a Opção B é a alternativa.

Opção B. Ollama em EC2 apenas com CPU

Funciona para modelos de 1B a 3B com performance aceitável (cerca de 3 a 10 tokens por segundo) para fins acadêmicos. Sugere-se usar t3.large ou t3.xlarge (8 a 16 GB de RAM). Modelos recomendados nesse cenário: Llama 3.2 1B, Gemma 3 2B, Phi-3.5 Mini, SmolLM3 3B com quantização Q4_K_M. É lento, mas perfeitamente didático para demonstrar paralelismo: a equipe pode subir múltiplas instâncias t3 e mostrar que o throughput agregado escala com o número de workers, mesmo com cada worker individual sendo modesto. Isso é, inclusive, uma demonstração mais pura de paralelismo do que usar uma API rápida.

Opção C. vLLM em EC2 com GPU (para equipes mais avançadas)

vLLM oferece throughput cerca de duas vezes maior que Ollama através de PagedAttention e continuous batching. É mais complexo de configurar, mas é a escolha ideal se a equipe quiser explorar batching dinâmico como tema central de paralelismo, algo que conversa diretamente com a disciplina. Requer instância GPU (g5.xlarge ou superior) e conhecimento prévio de Docker.

Opção D. SageMaker JumpStart

Se o AWS Academy tiver SageMaker disponível mas não o Bedrock, é possível fazer deploy de modelos open-source (Llama, Mistral) como endpoints SageMaker via JumpStart, com poucos cliques. A equipe paga pelo tempo de endpoint ativo, então recomenda-se ligar e desligar conforme uso.

Plano B Final: Execução Local

Se nada na AWS funcionar a contento, modelos pequenos como Llama 3.2 1B, Gemma 3 2B ou SmolLM3 rodam em laptops modernos (8 GB de RAM mínimo) via Ollama. A equipe pode demonstrar a parte de paralelismo distribuído entre máquinas dos próprios membros usando ngrok ou Tailscale para expor as instâncias. Não é a solução mais elegante, mas garante que ninguém fica sem entregar o trabalho por questões de infraestrutura.

7.4 Adaptações por Tema

A maioria dos temas funciona bem com qualquer SLM, mas algumas observações:

- **Temas 1 e 4** (documentos longos e tradução): prefira modelos com janela de contexto grande, como Qwen 2.5 (32K) ou Llama 3.1 (128K).
- **Tema 2** (multi-agente): prefira modelos com bom suporte a function calling, como Llama 3.2 ou Mistral.
- **Tema 5** (RAG): além do SLM gerador, será necessário um modelo de embeddings. Sugestões open-source: nomic-embed-text ou bge-small-en-v1.5, ambos rodáveis via Ollama.
- **Tema 8** (benchmarking): este tema ganha ao usar SLM, porque o foco é justamente comparar estratégias de prompting com infraestrutura controlada.

8. Entregáveis

Cada equipe deverá entregar os seguintes itens ao final do prazo:

8.1 Repositório Git

- Código-fonte completo da solução.
- Infraestrutura como código (Terraform, CloudFormation ou CDK), pode ser em versão simplificada.
- Scripts de deploy e de teste.
- README com instruções claras para reprodução do experimento.
- Diretório de prompts versionados, separado do código de aplicação.

8.2 Documento Técnico (8 a 15 páginas)

- Arquitetura do sistema, com diagrama explicativo.
- Justificativa das escolhas de paralelismo e distribuição.
- Descrição detalhada da engenharia de contexto aplicada, com prompts em apêndice.
- Análise dos resultados, com gráficos de latência, throughput e custo de tokens.
- Discussão sobre tolerância a falhas e cenários testados.
- Limitações da solução e propostas de melhoria futura.

8.3 Apresentação

- Apresentação de 15 a 20 minutos, com demonstração ao vivo do sistema processando uma carga real.
- Slides claros, com foco em arquitetura, decisões e resultados.

8.4 Relatório Individual

- Documento curto (1 a 2 páginas) por aluno, descrevendo sua contribuição específica e os principais aprendizados pessoais ao longo da atividade.

9. Critérios de Avaliação

A nota final será composta pelos seguintes critérios:

Critério	Peso
Corretude e funcionamento do sistema	25 por cento
Aplicação de conceitos de Programação Distribuída e Paralela	25 por cento
Qualidade da engenharia de contexto e prompts	20 por cento
Análise experimental e métricas coletadas	15 por cento
Qualidade da documentação e da apresentação	10 por cento

Critério	Peso
Inovação ou complexidade extra	5 por cento

Bônus: equipes que optarem por SLM auto-hospedado (Seção 7) receberão 10 pontos adicionais no critério de inovação e complexidade, em reconhecimento ao desafio de gerenciar a infraestrutura de inferência. Em contrapartida, espera-se documentação adicional sobre o modelo escolhido, quantização utilizada, métricas de tokens por segundo por worker e análise sobre a interação entre o paralelismo da aplicação e o paralelismo interno do servidor de inferência.